FACILE PMS

Sistema de Gestión Hotelera

Documentación Técnica y Funcional Completa

Versión 5.0 — Documento Maestro

Hotel:	Apartamentos Facile
Dirección:	Calle 97 #21-62, El Chicó / Chapinero, Bogotá, Colombia
Stack:	Java 17 · Spring Boot 3.x · React 18 · PostgreSQL 15 · JWT · Docker
Arquitectura:	Microservicios con API Gateway (Spring Cloud Gateway)
Contexto:	Proyecto SENA — Análisis y Desarrollo de Software
Equipo:	López · Echeverri · Mayor · Chico · Parada
Inspiración:	OPERA (Oracle Hospitality) / ZEUS PMS
Fecha:	2025

1. Resumen Ejecutivo

FACILE PMS es un sistema web integral de gestión hotelera (PMS — Property Management System) diseñado específicamente para Apartamentos Facile, un apart-hotel ubicado en el norte de Bogotá, Colombia. El sistema reemplaza procesos manuales basados en Excel, WhatsApp y documentos físicos con una plataforma digital centralizada que opera en tiempo real.

El proyecto se divide en dos fases de entrega: la Beta (Fase 1), que cubre el flujo operativo esencial con 7 microservicios, y la Versión Pro (Fase 2), que agrega módulos de negocio avanzado, integración con la DIAN, OTAs y reportes ejecutivos.

La arquitectura seleccionada es microservicios con Spring Boot, respaldada por un API Gateway centralizado, bases de datos PostgreSQL independientes por servicio y despliegue mediante Docker Compose. El frontend está construido en React 18 con Ant Design como sistema de componentes UI.

Esta documentación constituye el documento técnico maestro del proyecto y consolida la información de todas las versiones previas, añadiendo justificaciones técnicas, decisiones de diseño, historias de usuario, casos de uso y un plan de trabajo detallado fase por fase.

2. Descripción General del Proyecto

2.1 Contexto del Negocio

Apartamentos Facile es un apart-hotel boutique ubicado en Calle 97 #21-62, en el sector de El Chicó / Chapinero, uno de los corredores empresariales y residenciales más importantes del norte de Bogotá. El hotel combina dos tipos de unidades: apartamentos amoblados de largo y mediano plazo, y habitaciones de hotel para estadías cortas.

Campo	Detalle
Tipo de negocio	Apart-hotel — apartamentos amoblados + habitaciones de hotel
Dirección	Calle 97 #21-62, El Chicó / Chapinero, Bogotá
Contacto	+57 315 351 2085 · Español e inglés
Unidades totales	27 apartamentos + 10 habitaciones de hotel (37 unidades)
Check-in	3:00 p.m. – 10:00 p.m.
Check-out	6:00 a.m. – 1:00 p.m.
Recepción	24 horas al día, 7 días a la semana
Desayuno	Continental · Lunes a viernes · 6:30 – 9:00 a.m.

2.2 Unidades y Tarifas

Tipo de Unidad	Precio/noche	Capacidad	Distribución
Apartamento Dúplex 1 hab.	$250.000 COP	2 personas	1 hab · 1.5 baños · dúplex
Apartamento 2 habitaciones	$350.000 COP	4 personas	2 hab · 2 baños
Apartamento 3 habitaciones	$450.000 COP	6 personas	3 hab · 2 baños
Habitación de hotel	Desde $207.000 COP	1–2 personas	1 hab · baño privado

2.3 Servicios Disponibles

Servicio	Descripción	Cobro
Desayuno continental	Incluido en reservas L–V 6:30–9:00	Incluido
Restaurante	Almuerzos, cenas y consumos adicionales	Adicional
Centro de negocios	2 salas + cubículos privados · 24h	Consultar
Salas / eventos	Reuniones empresariales, eventos especiales	Por evento
Oficinas (3 tipos)	Tipo 1 y 3: dúplex 2 pers · Tipo 2: 4 pers	Consultar

2.4 Problema Actual y Justificación del Sistema

El hotel gestiona actualmente sus operaciones mediante Excel, documentos físicos y WhatsApp. Este modelo presenta los siguientes problemas críticos:

- Reservas duplicadas (overbooking): sin control centralizado, dos recepcionistas pueden asignar la misma unidad a distintos huéspedes.
- Inconsistencias en facturación: los consumos adicionales (restaurante, servicios) no se vinculan automáticamente a la factura del huésped.
- Falta de control de ocupación en tiempo real: no hay visibilidad instantánea del estado de cada unidad.
- Descoordinación entre áreas: housekeeping, mantenimiento y recepción operan sin un sistema compartido.
- Sin auditoría: es imposible saber quién realizó un cambio o canceló una reserva.
- Sin reportes financieros confiables: gerencia no tiene datos consolidados de ingresos, ocupación ni canales de venta.

2.5 Objetivos del Sistema

Objetivo General

Diseñar e implementar un sistema web integral que centralice y automatice la gestión operativa, administrativa y financiera del hotel, con información en tiempo real y mejora en la experiencia del cliente.

Objetivos Específicos

1. Centralizar la información del hotel en una única plataforma digital accesible 24/7.
2. Automatizar el control de reservas y disponibilidad de unidades, eliminando el overbooking.
3. Digitalizar el proceso de check-in y check-out con un tiempo de ejecución objetivo de 5 minutos o menos.
4. Centralizar los canales de comunicación con huéspedes (Gmail, WhatsApp) desde el sistema.
5. Gestionar el inventario de insumos con alertas automáticas ante stock bajo.
6. Generar reportes operativos y financieros confiables para la toma de decisiones gerenciales.
7. Controlar housekeeping, mantenimiento y servicios en tiempo real desde una única interfaz.
8. Cumplir con la obligación legal colombiana de facturación electrónica ante la DIAN (Versión Pro).

3. ¿Qué es un PMS? Fundamentos Conceptuales

3.1 Definición de PMS

Un PMS (Property Management System) es un sistema de software integral que centraliza y automatiza la gestión operativa de una propiedad de alojamiento — ya sea un hotel, hostal, apart-hotel o resort. El PMS es el núcleo digital del hotel: todos los procesos operativos (reservas, check-in, check-out, facturación, housekeeping) pasan por él.

El término proviene del sector hotelero y se diferencia de otros sistemas por integrar en una sola plataforma funcionalidades que de otro modo requerirían múltiples herramientas independientes: un sistema de reservas, un sistema de facturación, un sistema de inventario y una herramienta de comunicaciones.

3.2 ¿Por qué un PMS y no un CaaS o SaaS genérico?

Un CaaS (Cloud as a Service) genérico o un SaaS de gestión empresarial (como un ERP o CRM) no está diseñado para los flujos específicos del sector hotelero. Un PMS incluye conceptos que no existen en otros dominios:

- El ciclo de vida completo de una reserva hotelera: PENDIENTE → CONFIRMADA → EN_EJECUCION → FINALIZADA.
- Estados de unidades en tiempo real: DISPONIBLE, OCUPADA, SUCIA, EN_LIMPIEZA, INSPECCIONADA, MANTENIMIENTO, FUERA_DE_SERVICIO.
- Optimistic locking para control de concurrencia: dos recepcionistas no pueden reservar la misma unidad al mismo tiempo.
- Night Audit: proceso contable automático de cierre del día operativo hotelero.
- Folio segmentado: la factura de un huésped puede dividirse por empresa, cuenta personal o representación.
- Gestión de housekeeping vinculada directamente al ciclo de check-out.

Usar un ERP genérico (SAP, Odoo) para un hotel requeriría tal nivel de personalización que el resultado sería más costoso y complejo que construir un PMS dedicado. FACILE adopta el enfoque correcto: un sistema diseñado desde el inicio para el dominio hotelero.

3.3 Referentes de Industria — OPERA y ZEUS

FACILE se inspira en dos PMS de referencia que representan el estándar de la industria:

OPERA (Oracle Hospitality)

OPERA es el PMS más utilizado en cadenas hoteleras internacionales (Hilton, Marriott, IHG). Es el estándar de facto en hoteles de 4 y 5 estrellas a nivel mundial. Sus conceptos de folios segmentados, perfiles avanzados de huésped, auditoría nocturna y gestión de listas de espera son los que FACILE adopta y adapta al contexto colombiano.

ZEUS PMS

ZEUS es el sistema de referencia en el mercado hotelero latinoamericano, especialmente en Colombia y México. Ofrece módulos específicos para la normativa colombiana (facturación DIAN, SIRE para huéspedes extranjeros) que FACILE incorpora en su Versión Pro.

Concepto OPERA/ZEUS	Adoptado en FACILE	Cómo
Guest History / Perfiles	Sí	Tabla preferencia_cliente (tipo + valor: piso, cama, dieta, alergias)
Folios segmentados	Parcial	Campo folio_tipo en factura: PRINCIPAL / EMPRESA / PERSONAL
Políticas de cancelación	Sí	Tabla politica_cancelacion con horas_anticipacion y % penalidad
Lista de espera (Waitlist)	Sí	Estado EN_LISTA_ESPERA en estado_reserva
FUERA_DE_SERVICIO	Sí	Estado adicional en estado_unidad (diferente de MANTENIMIENTO)
Auditoría antes/después	Sí	Campos valor_anterior y valor_nuevo en tabla auditoria
Consumo vinculado a estadía	Sí	Campo estadia_id en consumo_servicio
Night Audit	Pro	@Scheduled en Spring Boot — cierre contable a medianoche
Revenue Management	Fase 2	Yield management y tarifas dinámicas por ocupación
Integración OTA	Pro	Webhooks de Booking.com, Expedia — ms-ota
Facturación electrónica	Pro	Integración DIAN en formato UBL 2.1 — ms-dian

4. Decisiones de Arquitectura — Justificación Completa

4.1 ¿Por qué Microservicios y no Monolito?

Esta es la decisión arquitectónica más importante del proyecto y merece una justificación detallada. Un monolito es una aplicación donde toda la lógica de negocio vive en un solo proyecto desplegable. Los microservicios dividen esa lógica en servicios pequeños, independientes, cada uno con su propia base de datos y ciclo de despliegue.

Argumentos a favor de Microservicios en FACILE

- Escalabilidad independiente: en temporada alta, ms-reservas y ms-checkin reciben mucho más tráfico que ms-housekeeping. Con microservicios, se puede escalar solo lo que se necesita, sin duplicar toda la aplicación.
- Despliegue independiente: si hay un bug en ms-facturacion, solo ese servicio se detiene y actualiza. El hotel sigue operando reservas y check-ins sin interrupciones.
- Aislamiento de fallos: un error en ms-reportes no afecta las operaciones críticas de ms-reservas.
- Distribución del trabajo: cada integrante del equipo trabaja en un microservicio independiente sin conflictos de merge en Git.
- Preparación para la Versión Pro: la arquitectura de microservicios permite agregar ms-inventario, ms-dian, ms-ota sin modificar ningún servicio existente. Un monolito requeriría refactorización extensa.
- Base de datos por servicio: ms-reservas tiene su propia base de datos aislada. Si ms-facturacion tiene un problema de BD, no afecta las reservas. Esto es imposible en un monolito con una sola BD.

Argumentos que se consideraron para el Monolito

- Menor complejidad inicial: un monolito es más simple de desarrollar al principio.
- Sin overhead de red: las llamadas entre módulos son directas en memoria, sin latencia de red.
- Menor número de contenedores Docker: un monolito despliega un solo contenedor.

Decisión Final: Microservicios desde el Inicio

FACILE elige microservicios desde la Beta porque el costo de migrar de un monolito a microservicios más adelante es significativamente mayor que construir la infraestructura correcta desde el principio. El equipo invierte una vez en la infraestructura y no tiene que reescribir nada para la Versión Pro.

> **Nota técnica:** La versión v4.0 del documento originalmente proponía un Monolito Modular. Tras análisis detallado, la arquitectura de microservicios es la correcta para este proyecto dadas las necesidades de escalabilidad, despliegue independiente y la división del trabajo en equipo. La Beta ya implementa microservicios.

4.2 Mapa de Microservicios — Beta

Servicio	Puerto	Base de Datos	Responsabilidad
API Gateway (Spring Cloud Gateway)	8080	—	Enrutamiento, JWT validation, Rate limiting
ms-auth	8081	db_auth	Autenticación JWT, roles, sesiones

Servicio	Puerto	Base de Datos	Responsabilidad
ms-clientes	8082	db_clientes	Registro y búsqueda de huéspedes
ms-unidades	8083	db_unidades	Inventario, estados, disponibilidad
ms-reservas	8084	db_reservas	Ciclo de vida completo de reservas
ms-checkin	8085	db_checkin	Check-in / Check-out transaccional
ms-housekeeping	8086	db_housekeeping	Tareas de limpieza post-checkout
ms-facturacion	8087	db_facturacion	Factura de noches y pago simple
React Frontend	3000	—	Interfaz de usuario del sistema

4.3 Mapa de Microservicios — Pro (adicionales)

Servicio	Puerto	Descripción
ms-pagos	8088	Pagos parciales, mixtos, anticipos, reembolsos
ms-inventario	8089	Control de insumos con alertas de stock
ms-mantenimiento	8090	Registro de fallas, asignación a técnicos
ms-servicios	8091	Consumos de restaurante, coworking, eventos
ms-reportes	8092	Dashboard ejecutivo y reportes exportables
ms-auditoria	8093	Auditoría completa con valor anterior/posterior
ms-configuracion	8094	Parámetros globales del sistema en tiempo real
ms-dian	8095	Facturación electrónica DIAN — UBL 2.1
ms-ota	8096	Integración Booking.com, Expedia, Airbnb

4.4 Comunicación entre Servicios

En la Beta, la comunicación entre microservicios se realiza mediante llamadas REST síncronas usando FeignClient de Spring Cloud OpenFeign. Esto es suficiente para la fase actual y no requiere infraestructura de mensajería asíncrona.

Llamada	Origen	Destino	Motivo
Validar token JWT	Todos los ms	ms-auth	Verificar sesión activa
Buscar cliente por ID	ms-reservas	ms-clientes	Asociar huésped a reserva
Verificar disponibilidad	ms-reservas	ms-unidades	Evitar doble reserva
Crear estadía	ms-checkin	ms-reservas	Confirmar reserva activa
Cambiar estado unidad	ms-checkin	ms-unidades	OCUPADA al hacer check-in
Crear tarea limpieza	ms-checkin	ms-housekeeping	Automático al hacer check-out
Crear factura	ms-checkin	ms-facturacion	Factura abierta al check-in

En la Versión Pro, algunos módulos como ms-auditoria y ms-reportes pueden usar mensajería asíncrona con eventos para evitar que las operaciones críticas dependan de servicios secundarios. Esto se evalúa con Apache Kafka o RabbitMQ en una fase posterior.

5. Stack Tecnológico — Justificación Técnica Detallada

Esta sección justifica cada tecnología elegida explicando por qué es la mejor opción para FACILE y por qué se descartaron las alternativas. Todas las herramientas seleccionadas son de código abierto y gratuitas para uso educativo y comercial.

5.1 Backend: Java 17 + Spring Boot 3.x

¿Por qué Java?

- Tipado estático fuerte: Java obliga a declarar tipos de datos explícitamente, lo que previene errores en tiempo de compilación que en lenguajes dinámicos solo aparecen en producción.
- JVM (Java Virtual Machine): compilar una vez, ejecutar en cualquier sistema operativo. El mismo .jar corre en Windows, Linux y Mac sin cambios.
- Ecosistema empresarial maduro: Spring Boot, Hibernate, Maven — herramientas con más de 15 años de uso en producción en sistemas bancarios, hoteleros y de retail.
- Rendimiento: la JVM moderna con JIT compilation es tan rápida como C++ en escenarios de servidor de larga duración (warm JVM).
- @Transactional de Spring: garantiza atomicidad en operaciones críticas como check-in (que modifica 4 tablas simultáneamente). Si falla cualquier paso, se revierten todos los cambios.
- Seguridad empresarial: Spring Security es el framework de seguridad más completo del ecosistema, con soporte nativo para JWT, OAuth2, LDAP y RBAC (Role-Based Access Control).

¿Por qué no Python?

- Python usa tipado dinámico, lo que significa que errores de tipo solo se detectan en tiempo de ejecución. Para un sistema financiero como un PMS, esto es inaceptable.
- El ecosistema hotelero y bancario usa Java. Los integradores de OPERA, ZEUS y la DIAN ofrecen SDKs en Java primero.
- FastAPI o Django no tienen un equivalente a Spring Security con la misma madurez para sistemas con múltiples roles y permisos complejos.

¿Por qué no Node.js / JavaScript?

- Node.js es excelente para APIs simples y en tiempo real, pero su naturaleza asíncrona basada en callbacks/promesas hace que las transacciones complejas (como el check-in que modifica 4 entidades) sean más difíciles de manejar correctamente.
- La ausencia de tipado en JavaScript vanilla (incluso con TypeScript) hace que refactorizaciones grandes sean más riesgosas.
- Spring Boot ofrece más herramientas de producción: actuator, micrometer, circuit breakers — sin dependencias adicionales.

¿Por qué no Go o Rust?

- Go y Rust son excelentes para microservicios de alto rendimiento, pero el ecosistema de librerías para el dominio hotelero (DIAN, OTAs, POS) es casi inexistente en estos lenguajes.
- El equipo de FACILE tiene formación en Java, lo que reduce la curva de aprendizaje a cero.

¿Por qué Spring Boot y no otro framework Java?

- Spring Boot incluye servidor embebido (Tomcat), configuración automática y starter dependencies — no requiere configuración de XML.

- Spring Cloud Gateway, Spring Data JPA, Spring Security son parte del mismo ecosistema y se integran perfectamente.
- @Transactional, @PreAuthorize, @Version — anotaciones que implementan comportamientos complejos con una sola línea de código.
- Alternativas como Quarkus o Micronaut son más rápidas en startup, pero Spring Boot tiene mayor documentación, comunidad y ejemplos en español.

5.2 Base de Datos: PostgreSQL 15

¿Por qué PostgreSQL?

- ACID compliant: PostgreSQL garantiza Atomicidad, Consistencia, Aislamiento y Durabilidad — esencial para operaciones financieras y de reservas.
- Columnas GENERATED: PostgreSQL permite columnas calculadas automáticamente en el servidor (subtotal = cantidad * precio_unitario). Esto garantiza que los cálculos son siempre correctos, independientemente de quién inserte el dato.
- Optimistic locking nativo: la columna version INT + @Version de JPA funciona de forma nativa con PostgreSQL.
- Índices parciales y expresiones: CREATE INDEX ON reserva(unidad_id) WHERE activo = TRUE — índices más eficientes.
- Tipos de datos ricos: NUMERIC(10,2) para precios (sin errores de punto flotante), DATE para fechas, TIMESTAMP para marcas de tiempo.
- Totalmente gratuito: PostgreSQL es open source con licencia PostgreSQL License, sin restricciones comerciales.

¿Por qué no MySQL?

- MySQL históricamente tiene soporte limitado para columnas GENERATED ALWAYS, que FACILE usa en factura_detalle.subtotal.
- El modo por defecto de MySQL tiene validaciones de integridad más permisivas que PostgreSQL, lo que puede resultar en datos inconsistentes.
- PostgreSQL cumple el estándar SQL más fielmente, lo que facilita migraciones futuras.

¿Por qué no MongoDB u otras NoSQL?

- Los datos de un PMS son profundamente relacionales: una reserva tiene cliente, unidad, tarifa, canal, política de cancelación. Las relaciones son parte del modelo de negocio.
- Las bases de datos de documentos (MongoDB) no garantizan integridad referencial de forma nativa. En un sistema financiero, las FK (Foreign Keys) son la primera línea de defensa contra datos huérfanos.
- Las consultas de disponibilidad (¿qué unidades están libres entre el 15 y el 20 de marzo?) son joins SQL complejos que son naturales en PostgreSQL y complicados en MongoDB.

¿Por qué no SQLite?

- SQLite no soporta concurrencia real: solo un proceso puede escribir a la vez. En un sistema hotelero con múltiples recepcionistas, esto sería inaceptable.

5.3 Frontend: React 18

¿Por qué React?

- Componentes reutilizables: la interfaz hotelera tiene muchos elementos repetidos (tablas de reservas, formularios de cliente, tarjetas de estado de unidad). React permite definirlos una vez y reutilizarlos.
- Virtual DOM: React actualiza solo los componentes que cambian, no toda la página. Esto es crítico para el dashboard en tiempo real que muestra estado de 37 unidades.
- Ecosistema maduro: Ant Design, React Router, React Hook Form, Zustand, Recharts — todas las herramientas necesarias tienen soporte de primera clase en React.
- JSX: combinar HTML y lógica JavaScript en el mismo archivo facilita el desarrollo de componentes complejos como el calendario de disponibilidad.
- Mercado laboral: React es el framework frontend más demandado en Colombia y Latinoamérica.

¿Por qué no Vue.js?

- Vue.js es una excelente alternativa. La razón para elegir React es el ecosistema de componentes hoteleros: Ant Design tiene componentes específicos para tablas de datos, calendarios de fechas y formularios complejos que son exactamente lo que FACILE necesita.

¿Por qué no Angular?

- Angular impone una estructura más rígida y tiene una curva de aprendizaje más pronunciada. Para un equipo en formación, React ofrece la mejor relación entre poder y simplicidad.

¿Por qué no HTML + CSS + JavaScript vanilla?

- Sin un framework, manejar el estado de 37 unidades en tiempo real, formularios con validaciones complejas y rutas protegidas por rol requeriría implementar manualmente lo que React, Zustand y React Router ya ofrecen.

5.4 Seguridad: JWT + Spring Security

¿Por qué JWT?

JWT (JSON Web Token) es un estándar abierto (RFC 7519) para transmitir información de autenticación de forma segura y compacta. En FACILE, el token JWT contiene el rol del empleado (ADMIN, RECEPCION, HOUSEKEEPING) y su número de cédula, lo que permite que cualquier microservicio valide la identidad y permisos del usuario sin consultar la base de datos en cada petición.

- Sin estado (stateless): cada petición lleva su propio token. Los microservicios no necesitan compartir sesiones en memoria ni acceder a Redis.
- Auto-contenido: el token incluye rol, cédula y fecha de expiración. El microservicio lee esta información directamente del token.
- Firma digital: el token está firmado con una clave secreta (JWT_SECRET). Si alguien intenta modificar el rol del token, la firma se invalida y el servidor lo rechaza.

¿Por qué no sesiones tradicionales (cookies)?

- Las sesiones tradicionales requieren que todos los microservicios compartan el mismo almacén de sesiones (Redis, base de datos centralizada). Esto introduce una dependencia central que elimina el beneficio del aislamiento de microservicios.
- JWT permite escalar horizontalmente sin sincronización de estado.

¿Por qué no OAuth2 / OpenID Connect?

- OAuth2 es el estándar correcto para permitir que aplicaciones de terceros accedan al sistema (ej: el hotel integrándose con Booking.com). En la Versión Pro, ms-ota usará OAuth2.
- Para la autenticación interna de empleados, JWT es más simple y directo que OAuth2, que requiere un servidor de autorización adicional (Keycloak, Auth0).

5.5 Contenedores: Docker + Docker Compose

¿Por qué Docker?

- Reproducibilidad: 'en mi máquina sí funciona' desaparece. Docker garantiza que el entorno de desarrollo, pruebas y producción son idénticos.
- Aislamiento: cada microservicio vive en su propio contenedor. ms-auth no puede interferir con los recursos del sistema de ms-facturacion.
- Un solo comando: docker compose up --build levanta los 11+ contenedores (7 BDs, 7 microservicios, API Gateway, frontend) en menos de 3 minutos.
- Volúmenes persistentes: los datos de PostgreSQL sobreviven al reinicio de contenedores gracias a los volúmenes Docker.

¿Por qué no máquinas virtuales (VMs)?

- Una VM requiere un sistema operativo completo por instancia (2–4 GB RAM por VM). Un contenedor Docker comparte el kernel del host y usa solo los recursos que necesita (típicamente 256–512 MB por microservicio).
- El startup de un contenedor es de segundos; el de una VM es de minutos.

5.6 API Gateway: Spring Cloud Gateway

El API Gateway es el punto de entrada único para todas las peticiones del frontend. Actúa como enrutador, validador de JWT y capa de rate limiting. Sin un API Gateway, el frontend tendría que conocer la dirección de cada microservicio, lo que complica el mantenimiento.

- Enrutamiento centralizado: /api/v1/reservas/* → ms-reservas:8084, /api/v1/clientes/* → ms-clientes:8082.
- Validación JWT centralizada: los tokens se validan en el gateway antes de llegar a los microservicios, reduciendo la carga de validación en cada servicio.
- Rate limiting: previene ataques de fuerza bruta al login limitando el número de peticiones por IP.
- Spring Cloud Gateway se integra nativamente con Spring Boot sin configuración adicional.

5.7 Otras Tecnologías del Stack

Tecnología	Categoría	Justificación
Maven 3.9+	Build tool	Gestión de dependencias declarativa en pom.xml. Estándar en el ecosistema Spring.
JPA / Hibernate	ORM	Mapea clases Java a tablas PostgreSQL. @Entity, @ManyToOne, @Version — sin SQL manual para operaciones CRUD.

Tecnología	Categoría	Justificación
Bean Validation	Validación	@NotNull, @Email, @Size en DTOs. Valida datos en la capa de entrada antes de llegar a la BD.
Springdoc OpenAPI	Docs API	Genera Swagger UI automáticamente desde anotaciones. Accesible en /swagger-ui.html.
Lombok	Boilerplate	@Getter, @Setter, @Builder reducen código repetitivo en entidades y DTOs.
Ant Design	UI Library	Componentes React enterprise: tablas, formularios, modales, calendarios de fechas.
Zustand	State mgmt	Manejo del estado global (token JWT, rol, usuario) sin la complejidad de Redux.
React Hook Form	Formularios	Formularios con validación sin re-renders innecesarios. Ideal para check-in.
Axios	HTTP Client	Llamadas HTTP al API Gateway con interceptor automático del token JWT.
Recharts	Gráficas	Gráficas del dashboard de ocupación e ingresos construidas con React.
Day.js	Fechas	Manejo de fechas (check-in, check-out, rangos). Más liviano que Moment.js.
Postman	Testing API	Pruebas de endpoints REST durante el desarrollo. Colecciones por microservicio.
pgAdmin 4	DB Admin	Interfaz visual para administrar PostgreSQL. Verificar DDL y datos.
Git	Versiones	Control de versiones. Monorepo con todos los microservicios.

6. Módulos del Sistema — Beta (Fase 1)

La Beta cubre 7 módulos core en arquitectura de microservicios, implementando el flujo operativo esencial que permite al hotel operar digitalmente desde el primer día. La base de datos incluye las 33 tablas del DDL completo v4.0, dejando la estructura lista para la Versión Pro.

6.1 ms-auth — Seguridad y Autenticación

Responsable de la autenticación de empleados mediante cédula + contraseña. Emite tokens JWT firmados con una vigencia de 24 horas. Todos los demás microservicios validan el token consultando este servicio o verificando la firma JWT localmente con la clave compartida.

Elemento	Detalle
Puerto	8081
Base de datos	db_auth
Tablas propias	rol · usuario · usuario_rol
Login	cédula + contraseña (hash BCrypt)
Roles Beta	ADMIN · RECEPCION · HOUSEKEEPING
Roles Pro	CONTABILIDAD · GERENCIA · MANTENIMIENTO (se agregan en Pro)
Token	JWT firmado · 24h · incluye rol y cédula del usuario
Algoritmo hash	BCrypt con salt automático — irreversible y resistente a fuerza bruta

Flujo de Autenticación JWT Detallado

El siguiente flujo describe el proceso completo desde el login hasta el acceso a un endpoint protegido:

9. El recepcionista envía POST /api/v1/auth/login con { cedula: '12345678', password: 'pass123' }.
10. ms-auth busca al usuario por cédula en db_auth. Si no existe, devuelve 401 Unauthorized.
11. ms-auth verifica la contraseña con BCrypt.matches(). Si no coincide, devuelve 401.
12. ms-auth genera un JWT firmado con JWT_SECRET que incluye: sub (cédula), role (RECEPCION), iat (emisión), exp (expiración en 24h).
13. El frontend guarda el token en el estado global de Zustand (no en localStorage — seguridad XSS).
14. Cada petición posterior incluye el header Authorization: Bearer eyJ... .
15. El API Gateway valida la firma del token antes de enrutar al microservicio destino.
16. El microservicio destino lee el rol del token para @PreAuthorize — sin consultar la BD.

6.2 ms-clientes — Gestión de Huéspedes

Gestiona el registro y búsqueda de clientes del hotel. La búsqueda por cédula es la operación más frecuente en recepción — permite identificar huéspedes en segundos sin necesidad de conocer su ID interno.

Elemento	Detalle
Puerto	8082

Elemento	Detalle
Base de datos	db_clientes
Tablas propias	tipo_documento · cliente · historial_cliente · preferencia_cliente
Operaciones Beta	Crear · Listar · Buscar por cédula · Actualizar · Desactivar (soft delete)
Soft delete	activo = false en vez de DELETE — los datos nunca se pierden en producción
Búsqueda principal	GET /api/v1/clientes/cedula/{cedula} — O(log n) con índice en cédula

6.3 ms-unidades — Gestión de Unidades

Mantiene el inventario físico del hotel: los 27 apartamentos y 10 habitaciones. Gestiona sus estados en tiempo real y responde consultas de disponibilidad para fechas específicas.

Elemento	Detalle
Puerto	8083
Base de datos	db_unidades
Tablas propias	tipo_unidad · estado_unidad · unidad · tipo_tarifa · tarifa
Estados de unidad	DISPONIBLE · OCUPADA · SUCIA · EN_LIMPIEZA · INSPECCIONADA · MANTENIMIENTO · FUERA_DE_SERVICIO
Concurrencia	@Version JPA + version INT — optimistic locking para evitar doble reserva
Query crítica	SELECT unidades NO reservadas en un rango de fechas (disponibilidad)

Diferencia entre MANTENIMIENTO y FUERA_DE_SERVICIO

Esta distinción, inspirada en OPERA, es importante para los reportes de ocupación. MANTENIMIENTO significa que la unidad tiene un problema activo siendo atendido y se espera que vuelva a estar disponible en el corto plazo. FUERA_DE_SERVICIO indica que la unidad está retirada de operación indefinidamente (renovación, daño estructural), por lo que no debe aparecer en los cálculos de ocupación y disponibilidad.

6.4 ms-reservas — El Corazón del Sistema

Módulo central de la Beta. Gestiona el ciclo de vida completo de una reserva: creación, modificación, cancelación y cambios de estado. Implementa control de concurrencia para evitar overbooking cuando dos recepcionistas intentan reservar la misma unidad simultáneamente.

Elemento	Detalle
Puerto	8084
Base de datos	db_reservas
Tablas propias	canal_reserva · estado_reserva · reserva · reserva_huesped · estadia · estado_estadia · politica_cancelacion

Elemento	Detalle
Estados de reserva	PENDIENTE · CONFIRMADA · EN_EJECUCION · CANCELADA · NO_SHOW · EN_LISTA_ESPERA
Concurrencia	@Version JPA — devuelve 409 Conflict si dos hilos intentan reservar la misma unidad
reserva_huesped	Permite múltiples huéspedes por reserva (grupos familiares, corporativos)
Lista de espera	Estado EN_LISTA_ESPERA cuando la unidad no está disponible en las fechas solicitadas
Canales	DIRECTA · OTA · AGENCIA · TELEFONO — trazabilidad de origen de reservas

Control de Concurrencia — Optimistic Locking

El escenario de concurrencia es real: en un hotel, dos recepcionistas pueden intentar reservar la misma unidad para las mismas fechas al mismo tiempo. Sin control de concurrencia, ambas reservas se confirman y el hotel tiene un overbooking.

FACILE usa optimistic locking con @Version JPA: la tabla reserva tiene una columna version INT. Cuando dos transacciones intentan modificar el mismo registro simultáneamente, la segunda transacción detecta que la versión cambió y Spring lanza OptimisticLockException, que se convierte en un 409 Conflict hacia el cliente.

6.5 ms-checkin — Check-in / Check-out

Orquesta el flujo de entrada y salida de huéspedes. Es el módulo con más interacciones entre servicios: un solo check-in actualiza estados en 4 servicios diferentes en una sola operación transaccional.

Elemento	Detalle
Puerto	8085
Base de datos	db_checkin
Check-in (una transacción)	reserva → EN_EJECUCION · estadia ACTIVO · unidad → OCUPADA · factura ABIERTA
Check-out (una transacción)	estadia → FINALIZADO · unidad → SUCIA · factura → PAGADA · tarea limpieza PENDIENTE
Tiempo objetivo	≤ 5 minutos por operación completa (RF-008)
@Transactional	Si cualquier paso falla, Spring revierte TODOS los cambios — datos siempre consistentes

6.6 ms-housekeeping — Limpieza

Gestiona el flujo de limpieza post-checkout. Cuando recepción ejecuta un check-out, este módulo recibe automáticamente una tarea de tipo POST_CHECKOUT.

Elemento	Detalle
Puerto	8086
Base de datos	db_housekeeping
Tablas propias	estado_limpieza · tipo_limpieza · limpieza
Flujo de estados	PENDIENTE → EN_PROCESO → COMPLETADA
Efecto en unidad	SUCIA → EN_LIMPIEZA → INSPECCIONADA → DISPONIBLE
Creación automática	ms-checkin crea tarea POST_CHECKOUT al ejecutar checkout
Tipos de limpieza	RUTINARIA · PROFUNDA · POST_CHECKOUT

6.7 ms-facturacion — Facturación Simple (Beta)

Cubre el mínimo necesario para cerrar el ciclo en la Beta: calcular el costo total de la estadía por noches y registrar un pago simple.

Elemento	Detalle
Puerto	8087
Base de datos	db_facturacion
Tablas propias	estado_factura · factura · factura_detalle · metodo_pago · estado_pago · pago
Cálculo Beta	precio_noche × cantidad_noches = total (sin IVA discriminado)
Pago Beta	Un único pago por factura (efectivo, tarjeta o transferencia)
Estados	ABIERTA (al check-in) → PAGADA (al check-out o pago manual)
factura_detalle.subtotal	Columna GENERATED ALWAYS — PostgreSQL la calcula automáticamente

7. Módulos del Sistema — Versión Pro (Fase 2)

La Versión Pro agrega 10 módulos nuevos o mejorados que convierten el sistema en un producto profesional y monetizable. Es prerrequisito que la Beta esté completamente operativa.

7.1 ms-facturacion (upgrade) — Facturación Completa

Elemento	Beta	Pro
Cálculo	precio × noches	precio × noches + servicios + IVA 19%
Folio	PRINCIPAL único	PRINCIPAL / EMPRESA / PERSONAL (segmentado)
Descuentos	No incluido	Descuento automático por cliente frecuente
Consumos	No incluido	Restaurante, coworking, eventos cargados a factura
Night Audit	No incluido	@Scheduled — cierre contable automático a medianoche
IVA	No discriminado	IVA 19% discriminado — leído de ms-configuracion en tiempo real

7.2 ms-pagos — Pagos Avanzados (Puerto 8088)

Elemento	Detalle
Pagos parciales	N pagos por factura hasta cubrir el total
Métodos mixtos	Efectivo + tarjeta en la misma factura
Anticipos	Pago antes del check-in que descuenta del total al checkout
Reembolsos	Estado REEMBOLSADO + registro del monto devuelto

7.3 ms-inventario — Control de Insumos (Puerto 8089)

Soporta entre 20 y 30 insumos (RNF-009). Alerta cuando stock_actual ≤ stock_minimo en ≤ 5 segundos.

Tablas nuevas	insumo (stock_actual, stock_minimo) · movimiento_inventario
Tipos de movimiento	ENTRADA · SALIDA · AJUSTE — registrados con usuario y fecha
Alerta de stock	Notificación en dashboard cuando stock cae por debajo del mínimo

7.4 ms-mantenimiento — Gestión de Fallas (Puerto 8090)

Al registrar una falla, cambia el estado de la unidad a MANTENIMIENTO y notifica al técnico en ≤ 15 segundos (RNF-012).

Elemento	Detalle
Tablas nuevas	estado_falla (REPORTADA, EN_PROCESO, RESUELTA, CERRADA) · falla
Rol nuevo	MANTENIMIENTO — usuario con acceso solo a este módulo
Notificación	spring-boot-starter-mail — email al técnico asignado en ≤ 15s

7.5 ms-servicios u2014 Servicios y Eventos (Puerto 8091)

Cada consumo queda vinculado a la estadía activa del huésped y se carga automáticamente a la factura abierta.

Elemento	Detalle
Tablas propias	tipo_servicio · servicio · consumo_servicio · estado_evento · evento
Tipos de servicio	RESTAURANTE · COWORKING · EVENTO · OTRO
Consumos	Al registrar → FeignClient a ms-facturacion agrega línea en factura_detalle
Eventos	RESERVADO → CONFIRMADO → FINALIZADO / CANCELADO

7.6 ms-reportes — Dashboard y Reportes (Puerto 8092)

Elemento	Detalle
Dashboard	Ocupación del día · Ingresos · Reservas activas · Alertas de stock · KPIs
Reportes	Ocupación por período · Ingresos por canal · Servicios · Housekeeping
Tiempo respuesta	Consultas simples ≤ 5s · Reportes complejos ≤ 45s (RNF-013)
WebSocket	spring-boot-starter-websocket para dashboard en tiempo real
Exportación	Apache POI (Excel) + OpenCSV — reportes descargables para gerencia

7.7 ms-auditoria — Auditoría Completa (Puerto 8093)

Registra automáticamente cada operación crítica con estado anterior y posterior.

Operaciones auditadas	Crear reserva · Cancelar reserva · Check-in · Check-out · Modificar cliente · Registrar pago · Cambiar estado
Formato	valor_anterior y valor_nuevo en JSON del estado completo de la entidad

7.8 ms-configuracion — Configuración del Sistema (Puerto 8094)

Centraliza parámetros globales. Cambiar un valor en la BD afecta el sistema en tiempo real sin redespliegue.

Parámetros clave
IVA_PORCENTAJE=19 · CHECKIN_HORA=15:00 · CHECKOUT_HORA=13:00 · HOTEL_NOMBRE · POLITICA_DEFAULT_ID

7.9 ms-dian — Facturación Electrónica (Puerto 8095)

Obligación legal para operación comercial en Colombia. Integra FACILE con la DIAN para emitir facturas válidas fiscalmente con CUFE (Código Único de Factura Electrónica). Se dispara automáticamente al cambiar factura a estado PAGADA.

Atención: ms-dian requiere registro como facturador electrónico ante la DIAN y credenciales de un proveedor tecnológico habilitado (Siigo, Alegra, Factus). Es la integración más compleja de la Pro.

7.10 ms-ota — Integración OTA / Channel Manager (Puerto 8096)

Conecta FACILE con Booking.com, Expedia y Airbnb mediante webhooks. Las reservas entrantes se crean automáticamente en ms-reservas con canal_reserva = OTA. Cuando cambia la disponibilidad, ms-ota notifica a las OTAs conectadas.

8. Flujos Operativos del Sistema

8.1 Flujo Operativo Completo — Ciclo de Reserva

El siguiente flujo describe el proceso completo desde que un huésped hace una reserva hasta que la unidad queda disponible para una nueva reserva:

17. LOGIN: El recepcionista se autentica con cédula + contraseña → ms-auth genera JWT → recepcionista accede al sistema.
18. REGISTRO DE CLIENTE: Si el huésped es nuevo, se registra con sus datos. Si ya existe, se busca por cédula en ms-clientes. La búsqueda por cédula usa el índice idx_cliente_cedula para respuesta en milisegundos.
19. CREAR RESERVA: El recepcionista selecciona fechas y unidad → ms-reservas verifica disponibilidad en ms-unidades → si está disponible, crea reserva en estado PENDIENTE.
20. CONFIRMAR RESERVA: Al registrar el anticipo o confirmación → reserva pasa a estado CONFIRMADA.
21. CHECK-IN (una sola transacción): El recepcionista ejecuta el check-in → ms-checkin ejecuta en una transacción atómica: (a) reserva → EN_EJECUCION, (b) estadía → ACTIVO, (c) unidad → OCUPADA, (d) factura → ABIERTA.
22. DURANTE LA ESTADÍA: El huésped puede consumir servicios adicionales (restaurante, coworking) → ms-servicios registra el consumo y lo agrega automáticamente a la factura abierta.
23. CHECK-OUT (una sola transacción): Al finalizar la estadía → ms-checkin ejecuta: (a) estadía → FINALIZADO, (b) factura → PAGADA, (c) unidad → SUCIA, (d) ms-housekeeping crea tarea POST_CHECKOUT automáticamente.
24. HOUSEKEEPING: El personal de limpieza actualiza el estado de la tarea: PENDIENTE → EN_PROCESO → COMPLETADA → la unidad pasa de SUCIA a INSPECCIONADA a DISPONIBLE.
25. UNIDAD DISPONIBLE: La unidad está lista para una nueva reserva. El ciclo comienza de nuevo.

8.2 Estados del Sistema

Estados de Reserva	Estados de Unidad	Estados de Estadía	Estados de Factura
PENDIENTE	DISPONIBLE	ACTIVO	ABIERTA
CONFIRMADA	OCUPADA	FINALIZADO	PAGADA
EN_EJECUCION	SUCIA	CANCELADO	CANCELADA
NO_SHOW	EN_LIMPIEZA	—	—
CANCELADA	INSPECCIONADA	—	—
EN_LISTA_ESPERA	MANTENIMIENTO	—	—
—	FUERA_DE_SERVICIO	—	—

8.3 Diagrama de Flujo — Check-in Detallado

El proceso de check-in es la operación más crítica del sistema. La anotación @Transactional garantiza que si cualquier paso falla (por ejemplo, ms-facturacion no responde), todos los cambios se revierten y la reserva queda en estado previo.

26. Recepcionista selecciona reserva CONFIRMADA y ejecuta check-in.
27. ms-checkin verifica que la reserva existe y está en estado CONFIRMADA.
28. ms-checkin llama a ms-reservas: actualizar estado → EN_EJECUCION.
29. ms-checkin crea registro en estadia con estado ACTIVO y fecha_checkin = NOW().
30. ms-checkin llama a ms-unidades: actualizar estado → OCUPADA.
31. ms-checkin llama a ms-facturacion: crear factura en estado ABIERTA con el cálculo de noches.
32. Si todos los pasos tienen éxito: transacción COMMIT. El check-in está completo.
33. Si cualquier paso falla: transacción ROLLBACK automático. La reserva vuelve a CONFIRMADA, la unidad sigue DISPONIBLE, no se crea factura.

9. Base de Datos — DDL Completo PostgreSQL v4.0

El modelo de datos consta de 33 tablas distribuidas en 9 módulos. Cada microservicio tiene su propia base de datos PostgreSQL independiente. Las tablas del DDL se dividen en 7 scripts init.sql, uno por microservicio.

9.1 Principios de Diseño

- Soft delete selectivo: las entidades operativas (cliente, unidad, reserva, servicio) usan activo BOOLEAN DEFAULT TRUE. Nunca se ejecuta DELETE en producción — los datos son auditables permanentemente.
- Optimistic locking: columna version INT DEFAULT 0 en reserva y unidad, usada con @Version de JPA para prevenir concurrencia sin locks pesimistas.
- Estados como tablas FK: todos los estados (reserva, estadía, limpieza, factura, unidad) son tablas independientes con FK. Nunca VARCHAR libre — imposible tener 'activo' vs 'ACTIVO' como valores distintos.
- Timestamps estándar: fecha_creacion y fecha_actualizacion en todas las entidades transaccionales. Trazabilidad completa.
- Índices de rendimiento: índices en cédula, fechas de reserva, unidad_id, factura_id, estadía para consultas rápidas.
- reserva_huesped (N:M): permite múltiples huéspedes por reserva, con campo es_titular para identificar al responsable.
- factura_detalle.subtotal GENERATED: PostgreSQL calcula automáticamente subtotal = cantidad * precio_unitario. Integridad matemática garantizada.

9.2 DDL — Módulo 1: Seguridad y Usuarios

```
-- 1. SEGURIDAD Y USUARIOS
CREATE TABLE rol (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE,
  descripcion TEXT
);

CREATE TABLE usuario (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  cedula VARCHAR(20) NOT NULL UNIQUE,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,        -- BCrypt hash
  activo BOOLEAN DEFAULT TRUE,
  ultimo_login TIMESTAMP,
  fecha_creacion TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW()
);

CREATE TABLE usuario_rol (
  usuario_id INT NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
  rol_id INT NOT NULL REFERENCES rol(id) ON DELETE CASCADE,
  PRIMARY KEY (usuario_id, rol_id)
);
```

```
CREATE TABLE auditoria (
  id SERIAL PRIMARY KEY,
  usuario_id INT REFERENCES usuario(id) ON DELETE SET NULL,
  accion VARCHAR(50) NOT NULL,
  entidad VARCHAR(100) NOT NULL,
  entidad_id INT,
  detalle TEXT,
  valor_anterior TEXT,      -- JSON del estado anterior
  valor_nuevo TEXT,         -- JSON del estado nuevo
  ip_address VARCHAR(45),
  fecha TIMESTAMP DEFAULT NOW()
);
```

¿Por qué usuario_rol como tabla N:M? Un empleado puede tener múltiples roles simultáneamente (ADMIN + CONTABILIDAD). Un rol puede pertenecer a múltiples usuarios. La tabla intermedia implementa este patrón correctamente.

9.3 DDL — Módulo 2: Clientes

```
CREATE TABLE tipo_documento (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE  -- CC, CE, PASAPORTE, NIT
);

CREATE TABLE cliente (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  tipo_documento_id INT REFERENCES tipo_documento(id),
  cedula VARCHAR(30) NOT NULL UNIQUE,
  telefono VARCHAR(20),
  email VARCHAR(150),
  nacionalidad VARCHAR(100),
  fecha_nacimiento DATE,
  activo BOOLEAN DEFAULT TRUE,       -- soft delete
  fecha_creacion TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW(),
  fecha_registro TIMESTAMP DEFAULT NOW()
);

CREATE TABLE preferencia_cliente (
  id SERIAL PRIMARY KEY,
  cliente_id INT NOT NULL REFERENCES cliente(id) ON DELETE CASCADE,
  tipo_preferencia VARCHAR(50) NOT NULL,   -- piso, tipo_cama, dieta
  valor VARCHAR(200) NOT NULL,
  observacion TEXT
);

CREATE TABLE historial_cliente (
  id SERIAL PRIMARY KEY,
  cliente_id INT NOT NULL REFERENCES cliente(id),
  usuario_id INT REFERENCES usuario(id),
  observacion TEXT NOT NULL,
```

```
  fecha TIMESTAMP DEFAULT NOW()
);
```

9.4 DDL — Módulo 3: Gestión de Unidades

```
CREATE TABLE tipo_unidad (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE
  -- DUPLEX_1HAB, APTO_2HAB, APTO_3HAB, HABITACION_HOTEL
);

CREATE TABLE estado_unidad (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- DISPONIBLE, OCUPADA, SUCIA, EN_LIMPIEZA,
  -- INSPECCIONADA, MANTENIMIENTO, FUERA_DE_SERVICIO
);

CREATE TABLE unidad (
  id SERIAL PRIMARY KEY,
  numero VARCHAR(10) NOT NULL UNIQUE,
  piso INT NOT NULL,
  tipo_unidad_id INT REFERENCES tipo_unidad(id),
  capacidad_adultos INT NOT NULL,
  capacidad_ninos INT NOT NULL,
  estado_unidad_id INT REFERENCES estado_unidad(id),
  bloqueo_administrativo BOOLEAN DEFAULT FALSE,
  descripcion TEXT,
  activo BOOLEAN DEFAULT TRUE,     -- soft delete
  fecha_creacion TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW(),
  version INT DEFAULT 0             -- optimistic locking @Version JPA
);

CREATE TABLE tipo_tarifa (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- ESTANDAR, CORPORATIVA, AGENCIA, PROMOCIONAL
);

CREATE TABLE tarifa (
  id SERIAL PRIMARY KEY,
  tipo_unidad_id INT REFERENCES tipo_unidad(id),
  tipo_tarifa_id INT REFERENCES tipo_tarifa(id),
  nombre_tarifa VARCHAR(100),
  precio NUMERIC(10,2) NOT NULL,
  fecha_inicio DATE,
  fecha_fin DATE
);
```

9.5 DDL — Módulo 4: Reservas y Estadías

```
CREATE TABLE canal_reserva (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- DIRECTA, OTA, AGENCIA, TELEFONO
);

CREATE TABLE estado_reserva (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- PENDIENTE, PAGO_REALIZADO, CONFIRMADA,
  -- EN_EJECUCION, NO_SHOW, CANCELADA, EN_LISTA_ESPERA
);

CREATE TABLE politica_cancelacion (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  horas_anticipacion INT NOT NULL,
  porcentaje_penalidad NUMERIC(5,2) DEFAULT 0,
  descripcion TEXT,
  activa BOOLEAN DEFAULT TRUE
);

CREATE TABLE reserva (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES cliente(id),
  unidad_id INT REFERENCES unidad(id),
  tarifa_id INT REFERENCES tarifa(id),
  canal_reserva_id INT REFERENCES canal_reserva(id),
  politica_cancelacion_id INT REFERENCES politica_cancelacion(id),
  fecha_inicio DATE NOT NULL,
  fecha_fin DATE NOT NULL,
  estado_reserva_id INT REFERENCES estado_reserva(id),
  cantidad_adultos INT DEFAULT 1,
  cantidad_ninos INT DEFAULT 0,
  observaciones TEXT,
  activo BOOLEAN DEFAULT TRUE,     -- soft delete
  fecha_creacion TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW(),
  version INT DEFAULT 0             -- optimistic locking
);

CREATE TABLE reserva_huesped (
  id SERIAL PRIMARY KEY,
  reserva_id INT NOT NULL REFERENCES reserva(id) ON DELETE CASCADE,
  cliente_id INT NOT NULL REFERENCES cliente(id),
  es_titular BOOLEAN DEFAULT FALSE
);

CREATE TABLE estado_estadia (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- ACTIVO, FINALIZADO, CANCELADO
);

CREATE TABLE estadia (
```

```
  id SERIAL PRIMARY KEY,
  reserva_id INT REFERENCES reserva(id),
  estado_estadia_id INT REFERENCES estado_estadia(id),
  fecha_checkin TIMESTAMP DEFAULT NOW(),
  fecha_checkout TIMESTAMP,
  usuario_checkin_id INT REFERENCES usuario(id),
  usuario_checkout_id INT REFERENCES usuario(id),
  fecha_creacion TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW()
);
```

9.6 DDL — Módulo 5: Limpieza

```
-- CORRECCIÓN v4.0: estado_limpieza como tabla FK (no VARCHAR)
CREATE TABLE estado_limpieza (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- PENDIENTE, EN_PROCESO, COMPLETADA
);

CREATE TABLE tipo_limpieza (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- RUTINARIA, PROFUNDA, POST_CHECKOUT
);

CREATE TABLE limpieza (
  id SERIAL PRIMARY KEY,
  unidad_id INT REFERENCES unidad(id),
  usuario_id INT REFERENCES usuario(id),
  tipo_limpieza_id INT REFERENCES tipo_limpieza(id),
  estado_limpieza_id INT REFERENCES estado_limpieza(id),  -- FK, no VARCHAR
  fecha TIMESTAMP DEFAULT NOW(),
  duracion_minutos INT,
  observacion TEXT
);
```

9.7 DDL — Módulo 6: Facturación y Pagos

```
CREATE TABLE estado_factura (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- ABIERTA, PAGADA, CANCELADA
);

CREATE TABLE factura (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES cliente(id),
  reserva_id INT REFERENCES reserva(id),
  estado_factura_id INT REFERENCES estado_factura(id),
  subtotal NUMERIC(10,2) NOT NULL DEFAULT 0,
  descuento NUMERIC(10,2) DEFAULT 0,
  impuesto NUMERIC(10,2) DEFAULT 0,         -- IVA 19%
```

```
  total NUMERIC(10,2) NOT NULL DEFAULT 0,
  folio_tipo VARCHAR(20) DEFAULT 'PRINCIPAL', -- PRINCIPAL/EMPRESA/PERSONAL
  fecha TIMESTAMP DEFAULT NOW(),
  fecha_actualizacion TIMESTAMP DEFAULT NOW()
);

CREATE TABLE factura_detalle (
  id SERIAL PRIMARY KEY,
  factura_id INT NOT NULL REFERENCES factura(id) ON DELETE CASCADE,
  tipo_concepto VARCHAR(50),
  descripcion VARCHAR(255) NOT NULL,
  cantidad NUMERIC(10,2) NOT NULL,
  precio_unitario NUMERIC(10,2) NOT NULL,
  subtotal NUMERIC(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED
);

CREATE TABLE metodo_pago (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- EFECTIVO, TARJETA, TRANSFERENCIA
);

CREATE TABLE estado_pago (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- PENDIENTE, PAGADO, REEMBOLSADO
);

CREATE TABLE pago (
  id SERIAL PRIMARY KEY,
  factura_id INT REFERENCES factura(id),
  metodo_pago_id INT REFERENCES metodo_pago(id),
  estado_pago_id INT REFERENCES estado_pago(id),
  monto NUMERIC(10,2) NOT NULL,
  referencia_pago VARCHAR(100),
  usuario_id INT REFERENCES usuario(id),
  fecha TIMESTAMP DEFAULT NOW()
);
```

9.8 DDL — Módulo 7: Servicios y Eventos

```
CREATE TABLE tipo_servicio (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- RESTAURANTE, COWORKING, EVENTO, OTRO
);

CREATE TABLE servicio (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  tipo_servicio_id INT REFERENCES tipo_servicio(id),
  precio_base NUMERIC(10,2),
  activo BOOLEAN DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT NOW()
```

```
);

CREATE TABLE consumo_servicio (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES cliente(id),
  servicio_id INT REFERENCES servicio(id),
  factura_id INT REFERENCES factura(id),
  estadia_id INT REFERENCES estadia(id),
  cantidad NUMERIC(10,2) DEFAULT 1,
  precio_unitario NUMERIC(10,2) NOT NULL,
  observacion TEXT,
  fecha TIMESTAMP DEFAULT NOW()
);

CREATE TABLE estado_evento (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- RESERVADO, CONFIRMADO, FINALIZADO, CANCELADO
);

CREATE TABLE evento (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(150) NOT NULL,
  cliente_id INT REFERENCES cliente(id),
  unidad_id INT REFERENCES unidad(id),
  estado_evento_id INT REFERENCES estado_evento(id),
  tipo_evento VARCHAR(100),
  fecha DATE NOT NULL,
  hora_inicio TIME,
  hora_fin TIME,
  cantidad_personas INT,
  precio NUMERIC(10,2),
  observaciones TEXT,
  fecha_creacion TIMESTAMP DEFAULT NOW()
);
```

9.9 DDL — Módulo 8: Configuración e Índices

```
CREATE TABLE configuracion (
  id SERIAL PRIMARY KEY,
  clave VARCHAR(50) NOT NULL UNIQUE,
  valor VARCHAR(255) NOT NULL,
  descripcion TEXT
  -- IVA_PORCENTAJE, CHECKIN_HORA, CHECKOUT_HORA,
  -- POLITICA_DEFAULT_ID, HOTEL_NOMBRE
);

-- ÍNDICES DE RENDIMIENTO
CREATE INDEX idx_reserva_fechas ON reserva(fecha_inicio, fecha_fin);
CREATE INDEX idx_reserva_unidad ON reserva(unidad_id);
CREATE INDEX idx_cliente_cedula ON cliente(cedula);
CREATE INDEX idx_usuario_cedula ON usuario(cedula);
CREATE INDEX idx_pago_factura ON pago(factura_id);
CREATE INDEX idx_estadia_reserva ON estadia(reserva_id);
```

```
CREATE INDEX idx_consumo_estadia ON consumo_servicio(estadia_id);
CREATE INDEX idx_reserva_huesped ON reserva_huesped(reserva_id);  -- v4.0
```

9.10 DDL Pro — Tablas Nuevas (Versión Pro)

```
-- ms-inventario
CREATE TABLE insumo (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  unidad_medida VARCHAR(30),
  stock_actual NUMERIC(10,2) DEFAULT 0,
  stock_minimo NUMERIC(10,2) DEFAULT 0,
  activo BOOLEAN DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT NOW()
);

CREATE TABLE movimiento_inventario (
  id SERIAL PRIMARY KEY,
  insumo_id INT REFERENCES insumo(id),
  tipo VARCHAR(20) NOT NULL,   -- ENTRADA, SALIDA, AJUSTE
  cantidad NUMERIC(10,2) NOT NULL,
  usuario_id INT NOT NULL,
  observacion TEXT,
  fecha TIMESTAMP DEFAULT NOW()
);

-- ms-mantenimiento
CREATE TABLE estado_falla (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
  -- REPORTADA, EN_PROCESO, RESUELTA, CERRADA
);

CREATE TABLE falla (
  id SERIAL PRIMARY KEY,
  unidad_id INT NOT NULL,
  descripcion TEXT NOT NULL,
  estado_falla_id INT REFERENCES estado_falla(id),
  tecnico_id INT,
  fecha_reporte TIMESTAMP DEFAULT NOW(),
  fecha_resolucion TIMESTAMP,
  observacion TEXT
);

-- ms-dian
CREATE TABLE factura_electronica (
  id SERIAL PRIMARY KEY,
  factura_id INT NOT NULL,
  cufe VARCHAR(200),
  estado_dian VARCHAR(50),
  respuesta_xml TEXT,
  fecha_emision TIMESTAMP DEFAULT NOW()
);
```

```
-- ms-ota
CREATE TABLE ota_reserva (
  id SERIAL PRIMARY KEY,
  reserva_externa_id VARCHAR(100) NOT NULL,
  plataforma VARCHAR(50),     -- BOOKING, EXPEDIA, AIRBNB
  reserva_id INT,
  sincronizado BOOLEAN DEFAULT FALSE,
  fecha TIMESTAMP DEFAULT NOW()
);
```

10. Contratos de API REST

Todos los endpoints están bajo el prefijo /api/v1. Las respuestas siguen el formato estándar JSON. Los endpoints protegidos requieren el header Authorization: Bearer <token>.

Formato de Respuesta Estándar

```
{
  "status": 200,
  "message": "OK",
  "data": { ... }
}
```

10.1 ms-auth — Autenticación

Método	Endpoint	Descripción	Rol
POST	/api/v1/auth/login	Login cédula + password → JWT	Público
POST	/api/v1/auth/logout	Invalida sesión	Autenticado
GET	/api/v1/auth/me	Datos del usuario autenticado	Autenticado

10.2 ms-clientes — Huéspedes

Método	Endpoint	Descripción	Rol
GET	/api/v1/clientes	Listar clientes activos	RECEPCION+
GET	/api/v1/clientes/{id}	Obtener cliente por ID	RECEPCION+
GET	/api/v1/clientes/cedula/{cedula}	Buscar por cédula	RECEPCION+
POST	/api/v1/clientes	Crear nuevo cliente	RECEPCION+
PUT	/api/v1/clientes/{id}	Actualizar datos	RECEPCION+
DELETE	/api/v1/clientes/{id}	Desactivar (soft delete)	ADMIN

10.3 ms-unidades

Método	Endpoint	Descripción	Rol
GET	/api/v1/unidades	Listar todas las unidades	Autenticado
GET	/api/v1/unidades/disponibles	Disponibles por fechas	Autenticado
GET	/api/v1/unidades/{id}	Detalle de una unidad	Autenticado
PUT	/api/v1/unidades/{id}/estado	Cambiar estado	RECEPCION+
POST	/api/v1/unidades	Crear unidad	ADMIN

10.4 ms-reservas

Método	Endpoint	Descripción	Rol
GET	/api/v1/reservas	Listar (filtros: fecha, estado, canal)	RECEPCION+
GET	/api/v1/reservas/{id}	Detalle de reserva	RECEPCION+
POST	/api/v1/reservas	Crear nueva reserva	RECEPCION+
PUT	/api/v1/reservas/{id}	Modificar reserva	RECEPCION+
PATCH	/api/v1/reservas/{id}/cancelar	Cancelar reserva	RECEPCION+

10.5 ms-checkin — Check-in / Check-out

Método	Endpoint	Descripción	Rol
POST	/api/v1/checkin/{reservaId}	Ejecutar check-in completo	RECEPCION+
POST	/api/v1/checkout/{estadiaId}	Ejecutar check-out completo	RECEPCION+
GET	/api/v1/estadias/activas	Ver estadías activas	RECEPCION+
GET	/api/v1/estadias/{id}	Detalle de estadía	RECEPCION+

10.6 ms-housekeeping

Método	Endpoint	Descripción	Rol
GET	/api/v1/limpieza	Listar tareas de limpieza	HOUSEKEEPING+
GET	/api/v1/limpieza/pendientes	Tareas pendientes	HOUSEKEEPING+
POST	/api/v1/limpieza	Crear tarea manual	RECEPCION+
PATCH	/api/v1/limpieza/{id}/estado	Cambiar estado de tarea	HOUSEKEEPING+

10.7 ms-facturacion

Método	Endpoint	Descripción	Rol
GET	/api/v1/facturas/{id}	Detalle de factura	RECEPCION+
GET	/api/v1/facturas/reserva/{id}	Factura de una reserva	RECEPCION+
POST	/api/v1/pagos	Registrar pago simple	RECEPCION+
GET	/api/v1/pagos/factura/{id}	Ver pago de una factura	RECEPCION+

10.8 Endpoints Adicionales — Versión Pro

Módulo	Método	Endpoint	Descripción
ms-pagos	POST	/api/v1/pagos	Pago parcial o total
ms-pagos	POST	/api/v1/pagos/{id}/reembolso	Registrar reembolso
ms-inventario	GET	/api/v1/insumos/alertas	Insumos bajo stock mínimo
ms-inventario	POST	/api/v1/insumos/{id}/movimiento	Registrar entrada/salida
ms-mantenimiento	POST	/api/v1/fallas	Reportar nueva falla
ms-servicios	POST	/api/v1/consumos	Registrar consumo de servicio
ms-servicios	GET	/api/v1/consumos/estadia/{id}	Consumos de una estadía
ms-reportes	GET	/api/v1/reportes/ocupacion	Ocupación por período
ms-reportes	GET	/api/v1/dashboard	KPIs del día en tiempo real
ms-auditoria	GET	/api/v1/auditoria	Listar eventos de auditoría
ms-configuracion	PUT	/api/v1/configuracion/{clave}	Actualizar parámetro

11. Infraestructura Docker

Docker garantiza que todos los miembros del equipo y cualquier servidor ejecuten exactamente el mismo entorno. docker-compose.yml orquesta todos los contenedores con un solo comando.

11.1 Estructura del Proyecto — Beta

```
facile-beta/
├── api-gateway/          ← Spring Cloud Gateway
│   ├── src/
│   ├── pom.xml
│   └── Dockerfile
├── ms-auth/              ← Seguridad y autenticación
│   ├── src/main/java/com/facile/msauth/
│   │   ├── controller/   ← @RestController
│   │   ├── service/      ← @Service, @Transactional
│   │   ├── repository/   ← JpaRepository
│   │   ├── entity/       ← @Entity
│   │   ├── dto/          ← DTOs con Bean Validation
│   │   ├── mapper/       ← entity <-> dto
│   │   └── client/       ← FeignClient a otros ms
│   ├── src/main/resources/
│   │   ├── application.properties
│   │   └── db/init.sql   ← DDL de las tablas de este servicio
│   ├── pom.xml
│   └── Dockerfile
├── ms-clientes/          ← (misma estructura que ms-auth)
├── ms-unidades/
├── ms-reservas/
├── ms-checkin/
├── ms-housekeeping/
├── ms-facturacion/
├── frontend/             ← React 18
│   ├── src/
│   ├── package.json
│   └── Dockerfile
├── docker-compose.yml
└── .env                  ← NO subir a Git
```

11.2 docker-compose.yml — Beta Completo

```
version: '3.8'
services:
  # ── Bases de datos ──────────────────────────
  db-auth:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: db_auth
      POSTGRES_USER: facile_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_auth_data:/var/lib/postgresql/data
      - ./ms-auth/src/main/resources/db/init.sql:
        /docker-entrypoint-initdb.d/init.sql
```

```
    networks: [facile-net]

  # (repetir para db-clientes:db_clientes, db-unidades:db_unidades,
  #  db-reservas:db_reservas, db-checkin:db_checkin,
  #  db-housekeeping:db_housekeeping, db-facturacion:db_facturacion)

  # ── API Gateway ──────────────────────────────
  api-gateway:
    build: ./api-gateway
    ports: ['8080:8080']
    environment:
      JWT_SECRET: ${JWT_SECRET}
    depends_on: [ms-auth, ms-clientes, ms-unidades,
      ms-reservas, ms-checkin, ms-housekeeping, ms-facturacion]
    networks: [facile-net]

  # ── Microservicios ───────────────────────────
  ms-auth:
    build: ./ms-auth
    ports: ['8081:8081']
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db-auth:5432/db_auth
      SPRING_DATASOURCE_USERNAME: facile_user
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
    depends_on: [db-auth]
    networks: [facile-net]

  # (repetir para ms-clientes:8082, ms-unidades:8083,
  #  ms-reservas:8084, ms-checkin:8085,
  #  ms-housekeeping:8086, ms-facturacion:8087)

  # ── Frontend ─────────────────────────────────
  frontend:
    build: ./frontend
    ports: ['3000:3000']
    environment:
      REACT_APP_API_URL: http://localhost:8080/api/v1
    depends_on: [api-gateway]
    networks: [facile-net]

volumes:
  db_auth_data:
  db_clientes_data:
  db_unidades_data:
  db_reservas_data:
  db_checkin_data:
  db_housekeeping_data:
  db_facturacion_data:

networks:
  facile-net:
    driver: bridge
```

11.3 Dockerfile Estándar — Microservicio Spring Boot

```
# Build stage
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

11.4 Dockerfile — Frontend React

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 3000
```

11.5 application.properties — Configuración por Microservicio

```
# Ejemplo: ms-auth/src/main/resources/application.properties
server.port=8081
spring.application.name=ms-auth

# Base de datos
spring.datasource.url=jdbc:postgresql://localhost:5432/db_auth
spring.datasource.username=facile_user
spring.datasource.password=facile_pass
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true

# Swagger
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html

# JWT
jwt.secret=${JWT_SECRET}
jwt.expiration=86400000     # 24 horas en milisegundos
```

```
# URLs de otros microservicios (para FeignClient)
ms.clientes.url=http://localhost:8082
ms.unidades.url=http://localhost:8083
ms.reservas.url=http://localhost:8084
ms.checkin.url=http://localhost:8085
ms.housekeeping.url=http://localhost:8086
ms.facturacion.url=http://localhost:8087
```

11.6 Comandos Esenciales Docker

Comando	Descripción
docker compose up --build	Construye todas las imágenes y levanta los 11+ contenedores
docker compose up -d	Levanta en segundo plano (detached mode)
docker compose down	Detiene y elimina los contenedores
docker compose down -v	Detiene, elimina contenedores y BORRA volúmenes (resetea BDs)
docker compose logs -f ms-auth	Ver logs de un microservicio en tiempo real
docker compose ps	Estado de todos los contenedores
docker exec -it facile-db-auth psql -U facile_user -d db_auth	Acceder a la BD directamente
docker compose restart ms-reservas	Reiniciar un microservicio sin afectar los demás

Seguridad crítica: El archivo .env debe contener JWT_SECRET y todas las contraseñas de BD. Agregar .env al .gitignore ANTES del primer commit. Nunca subir credenciales al repositorio de Git.

12. Dependencias e Instalación

12.1 Herramientas del Sistema

Herramienta	Versión	Por qué se necesita	Descarga
JDK	17 LTS o 21 LTS	Compila y ejecuta Spring Boot. Sin JDK no hay backend.	adoptium.net
Maven	3.9+	Gestiona dependencias del pom.xml. Incluido en IntelliJ.	Incluido en IntelliJ
Node.js	18 LTS o 20 LTS	Ejecuta React y npm. Sin Node.js no hay frontend.	nodejs.org
Docker Desktop	Última	Levanta toda la infraestructura con un solo comando.	docker.com
IntelliJ IDEA	Community/Ultimate	IDE principal para Java + Spring Boot.	jetbrains.com
Visual Studio Code	Última	IDE para el frontend React.	code.visualstudio.com
pgAdmin 4	Última	Interfaz visual para administrar las BDs PostgreSQL.	pgadmin.org
Postman	Última	Prueba endpoints REST. Guarda el token JWT para reutilizarlo.	postman.com
Git	2.x	Control de versiones. Monorepo con todos los servicios.	git-scm.com

12.2 Dependencias Backend — pom.xml por Microservicio

Dependencia (artifactId)	GroupId	Para qué sirve
spring-boot-starter-web	org.springframework.boot	Endpoints REST (@RestController)
spring-boot-starter-data-jpa	org.springframework.boot	Conexión con PostgreSQL vía JPA/Hibernate
spring-boot-starter-security	org.springframework.boot	Autenticación JWT y control de acceso por rol
spring-boot-starter-validation	org.springframework.boot	@NotNull, @Email, @Size en DTOs
postgresql	org.postgresql	Driver JDBC entre Spring Boot y PostgreSQL
jjwt-api / jjwt-impl / jjwt-jackson	io.jsonwebtoken	Generación y validación de tokens JWT (v0.12.x)
spring-cloud-starter-gateway	org.springframework.cloud	Solo en api-gateway: enrutamiento y filtros
spring-cloud-starter-openfeign	org.springframework.cloud	Llamadas REST entre microservicios
springdoc-openapi-starter-webmvc-ui	org.springdoc	Swagger UI automático (/{ms}/swagger-ui.html)

Dependencia (artifactId)	GroupId	Para qué sirve
lombok	org.projectlombok	@Getter, @Setter, @Builder — reduce código repetitivo
spring-boot-starter-test	org.springframework.boot	JUnit 5 + Mockito para pruebas
spring-boot-devtools	org.springframework.boot	Recarga automática al guardar cambios

12.3 Dependencias Frontend — package.json

Paquete npm	Instalación	Para qué sirve
react + react-dom	npx create-react-app facile-beta	Base del proyecto React 18
react-router-dom	npm install react-router-dom	Navegación entre vistas y rutas privadas por rol
axios	npm install axios	Llamadas HTTP al API Gateway con interceptor JWT automático
react-hook-form	npm install react-hook-form	Formularios de check-in, reservas, clientes
antd	npm install antd	Componentes UI: tablas, formularios, modales, calendarios
@ant-design/icons	npm install @ant-design/icons	Íconos de la interfaz
dayjs	npm install dayjs	Manejo de fechas en check-in, check-out, reservas
zustand	npm install zustand	Estado global: token JWT, usuario, rol activo
recharts	npm install recharts	Gráficas del dashboard (Pro)
socket.io-client	npm install socket.io-client	Dashboard en tiempo real vía WebSocket (Pro)

12.4 Dependencias Adicionales — Versión Pro

Dependencia	GroupId	Módulo	Para qué sirve
spring-boot-starter-quartz	org.springframework.boot	ms-facturacion	Night Audit automático a medianoche
spring-boot-starter-mail	org.springframework.boot	ms-mantenimiento	Notificaciones email al técnico (≤ 15s)
spring-boot-starter-websocket	org.springframework.boot	ms-reportes	Dashboard en tiempo real
poi-ooxml	org.apache.poi	ms-reportes	Exportar reportes a Excel (.xlsx)
opencsv	com.opencsv	ms-reportes	Exportar reportes a CSV

13. Requisitos Funcionales

ID	Descripción	Prioridad
RF-001	Gestión de contratos y documentos digitales con acceso por rol	MUST
RF-002	Control de ocupación y disponibilidad en tiempo real (27 aptos + 10 hab)	MUST
RF-003	Asignación y gestión de housekeeping, mantenimiento y atención por unidad	MUST
RF-004	Notificaciones automáticas de reservas y pagos por Gmail / WhatsApp	MUST
RF-005	Registro de apartamentos amoblados vinculados a servicios hoteleros	SHOULD
RF-006	Soporte técnico multicanal con historial de interacciones	SHOULD
RF-007	Clientes frecuentes: historial y descuentos aplicados automáticamente	MUST
RF-008	Check-in y check-out digital con tiempo de proceso ≤ 5 minutos	MUST
RF-009	Registro de fallas con asignación automática a técnicos	SHOULD
RF-010	Módulo financiero con vinculación automática de reservas y servicios	MUST
RF-011	Gestión de inventario con alertas automáticas de stock bajo	MUST
RF-012	Acceso 24/7 desde cualquier dispositivo con respuesta en tiempo real	MUST
RF-013	Interfaz responsive en Chrome, Safari, Edge, Firefox (móvil, tablet, PC)	SHOULD
RF-014	Reportes de servicios adicionales: restaurante, transporte, seguridad	SHOULD
RF-015	Gestión de No-Show con reglas automáticas de penalización	SHOULD
RF-016	Lista de espera (waitlist) para unidades en temporada alta	SHOULD
RF-017	Perfiles avanzados de huésped: preferencias, alergias, notas especiales	SHOULD
RF-018	Auditoría nocturna: cierre de día contable y reporte fin de día	SHOULD

14. Requisitos No Funcionales

ID	Descripción y Métrica
RNF-001	Rendimiento: operaciones estándar ≤ 3s · login ≤ 2s (base ~5.000 registros)
RNF-002	Seguridad: datos cifrados en tránsito (HTTPS) y reposo · permisos por rol · bcrypt
RNF-003	Disponibilidad: uptime mínimo 99.5% · accesible en móvil, tablet y PC
RNF-004	Escalabilidad: soportar 50–100 usuarios concurrentes sin retrasos > 5s
RNF-005	Temporada alta: tiempo de respuesta ≤ 6s con todas las funciones activas
RNF-006	Usabilidad: funciones básicas aprendibles en < 30 min sin capacitación técnica
RNF-007	Housekeeping: cambio de estado de unidad visible en ≤ 1.5s
RNF-008	Financiero: reportes sin errores · copias de seguridad diarias automáticas
RNF-009	Inventario: manejo de 20–30 insumos · alertas de stock en ≤ 5s
RNF-010	Comunicaciones: reservas y mensajes centralizados · disponibilidad ≥ 99.5%
RNF-011	Sistema: respuesta ≤ 3s · máx. 2h inactividad/mes
RNF-012	Mantenimiento: notificación al técnico en ≤ 15s con registro del incidente
RNF-013	Consultas simples ≤ 5s · reportes complejos ≤ 45s
RNF-014	Integridad: sin duplicados ni pérdidas ante fallos técnicos (@Transactional)
RNF-015	Docker: el entorno completo debe levantarse con docker compose up en ≤ 3 min
RNF-016	Swagger: documentación de la API accesible en /swagger-ui.html

15. Historias de Usuario

Las historias de usuario describen las funcionalidades del sistema desde la perspectiva del usuario final. Siguen el formato: 'Como [rol], quiero [funcionalidad] para [beneficio]'.

15.1 Rol: Recepcionista

ID	Historia de Usuario	Criterios de Aceptación
HU-001	Como recepcionista, quiero buscar un huésped por cédula para identificarlo rápidamente al llegar al hotel.	Buscar por cédula devuelve el perfil en < 2s. Si no existe, ofrece crear uno nuevo.
HU-002	Como recepcionista, quiero ver las unidades disponibles en un rango de fechas para asignar la correcta a una reserva.	La lista muestra unidades con estado DISPONIBLE y precio por noche. Filtra por fechas.
HU-003	Como recepcionista, quiero crear una reserva seleccionando cliente, unidad y fechas para registrar la llegada de un huésped.	La reserva se crea en estado PENDIENTE. Si hay concurrencia, el sistema devuelve 409.
HU-004	Como recepcionista, quiero ejecutar el check-in en máximo 5 minutos para atender al huésped rápidamente.	Check-in actualiza todos los estados en una sola operación. La factura se abre automáticamente.
HU-005	Como recepcionista, quiero ejecutar el check-out y registrar el pago para cerrar la estadía del huésped.	Check-out cierra la factura, cambia la unidad a SUCIA y crea tarea de limpieza automáticamente.
HU-006	Como recepcionista, quiero agregar consumos adicionales (restaurante, coworking) a la factura del huésped durante su estadía.	El consumo se agrega como línea en la factura abierta del huésped. El total se actualiza.
HU-007	Como recepcionista, quiero cancelar una reserva aplicando la política de cancelación correspondiente.	El sistema calcula la penalidad según horas_anticipacion y porcentaje_penalidad de la política.
HU-008	Como recepcionista, quiero poner a un huésped en lista de espera cuando la unidad deseada no está disponible.	La reserva se crea en estado EN_LISTA_ESPERA. Se notifica cuando se libera la unidad.

15.2 Rol: Personal de Housekeeping

ID	Historia de Usuario	Criterios de Aceptación
HU-009	Como personal de housekeeping, quiero ver la lista de unidades pendientes de limpieza para organizar mi turno.	La vista muestra tareas PENDIENTES con tipo (POST_CHECKOUT, RUTINARIA) y número de unidad.

ID	Historia de Usuario	Criterios de Aceptación
HU-010	Como personal de housekeeping, quiero actualizar el estado de una tarea de limpieza para indicar que terminé.	Cambiar estado PENDIENTE → EN_PROCESO → COMPLETADA. Al completar, la unidad pasa a DISPONIBLE.

15.3 Rol: Administrador

ID	Historia de Usuario	Criterios de Aceptación
HU-011	Como administrador, quiero registrar nuevos empleados con sus roles para controlar el acceso al sistema.	El nuevo usuario puede hacer login. Solo accede a las funciones de su rol.
HU-012	Como administrador, quiero desactivar un cliente sin eliminar su historial para mantener la trazabilidad.	Soft delete: activo=false. El historial de estadías previas se conserva.
HU-013	Como administrador, quiero ver el historial de acciones de cualquier usuario para auditar operaciones.	La tabla auditoria muestra accion, entidad, valor_anterior, valor_nuevo, ip y fecha.
HU-014	Como administrador, quiero cambiar la política de cancelación predeterminada sin necesidad de redesplegar el sistema.	Modificar POLITICA_DEFAULT_ID en ms-configuracion. El cambio es efectivo en tiempo real.

15.4 Rol: Gerencia (Pro)

ID	Historia de Usuario	Criterios de Aceptación
HU-015	Como gerente, quiero ver el dashboard de ocupación del día para tomar decisiones en tiempo real.	El dashboard muestra: unidades ocupadas/disponibles, ingresos del día, reservas activas y alertas.
HU-016	Como gerente, quiero generar un reporte de ingresos por canal para analizar qué canales son más rentables.	Reporte filtra por período y canal (DIRECTA, OTA, AGENCIA). Exportable a CSV/Excel.
HU-017	Como gerente, quiero recibir una notificación cuando el stock de un insumo baje del mínimo.	Alerta visible en el dashboard en ≤ 5s cuando stock_actual ≤ stock_minimo (RNF-009).

16. Casos de Uso Detallados

CU-001 — Crear Reserva con Control de Concurrencia

Campo	Detalle
Actor principal	Recepcionista
Precondición	Recepcionista autenticado. Cliente registrado en el sistema.
Flujo normal	1. Recepcionista selecciona cliente por cédula. 2. Selecciona fechas de entrada y salida. 3. Sistema consulta disponibilidad en ms-unidades. 4. Sistema muestra unidades disponibles con precios. 5. Recepcionista selecciona unidad. 6. Sistema crea reserva en estado PENDIENTE. 7. Recepcionista confirma y reserva pasa a CONFIRMADA.
Flujo alternativo A	Si la unidad fue reservada por otro recepcionista simultáneamente: sistema devuelve 409 Conflict. Recepcionista debe seleccionar otra unidad.
Flujo alternativo B	Si la unidad deseada no está disponible: sistema ofrece crear reserva en EN_LISTA_ESPERA.
Postcondición	Reserva creada en base de datos con estado CONFIRMADA. Índice idx_reserva_fechas actualizado.

CU-002 — Ejecutar Check-in

Campo	Detalle
Actor principal	Recepcionista
Precondición	Reserva en estado CONFIRMADA. Huésped presente en recepción.
Flujo normal	1. Recepcionista busca la reserva del huésped. 2. Selecciona 'Ejecutar Check-in'. 3. ms-checkin inicia @Transactional. 4. ms-reservas: reserva → EN_EJECUCION. 5. ms-checkin crea estadía con estado ACTIVO. 6. ms-unidades: unidad → OCUPADA. 7. ms-facturacion: crea factura ABIERTA con cálculo de noches. 8. Commit exitoso. Recepcionista ve confirmación.
Flujo de error	Si cualquier paso (3–7) falla: ROLLBACK automático. Reserva vuelve a CONFIRMADA. Sistema muestra error al recepcionista.
Tiempo objetivo	≤ 5 minutos desde que el huésped llega a recepción (RF-008).
Postcondición	Estadía ACTIVA. Unidad OCUPADA. Factura ABIERTA. Reserva EN_EJECUCION.

CU-003 — Flujo Completo de Housekeeping

Campo	Detalle
Actor principal	Personal de Housekeeping
Disparador	Checkout ejecutado en ms-checkin. ms-checkin crea tarea POST_CHECKOUT automáticamente.

Campo	Detalle
Flujo normal	1. Personal ve tarea POST_CHECKOUT en estado PENDIENTE. 2. Inicia limpieza: estado → EN_PROCESO. Unidad → EN_LIMPIEZA. 3. Termina limpieza: estado → COMPLETADA. 4. Supervisor inspecciona: unidad → INSPECCIONADA. 5. Sistema marca unidad → DISPONIBLE.
RNF asociado	Cambio de estado visible en ≤ 1.5s (RNF-007).
Postcondición	Unidad en estado DISPONIBLE. Lista para nueva reserva. Registro de duración_minutos en tabla limpieza.

17. Plan de Trabajo — Guía de Implementación Detallada

17.1 Beta — Fases 0 a 8

FASE 0 — Setup e Infraestructura

Paso	Descripción
0.1	Instalar en orden: JDK 17/21 → Docker Desktop → Node.js 18 → IntelliJ IDEA → VSCode → Git → pgAdmin 4 → Postman.
0.2	Crear carpeta raíz facile-beta/ con subcarpetas por microservicio. Inicializar git init. Crear .gitignore con .env ANTES del primer commit.
0.3	Dividir el DDL v4.0 en 7 archivos init.sql, uno por microservicio (ms-auth/src/main/resources/db/init.sql, etc.).
0.4	Crear docker-compose.yml con las 7 BDs + API Gateway + 7 microservicios + frontend.
0.5	Crear api-gateway con Spring Cloud Gateway con rutas vacías. Crear archivo .env con JWT_SECRET y DB_PASSWORD.
0.6	Ejecutar docker compose up --build. Verificar 11+ contenedores activos. Verificar tablas en pgAdmin.

FASE 1 — ms-auth (Seguridad)

- Entidades: Rol.java, Usuario.java, UsuarioRol.java (con @ManyToMany).
- Repositorios: UsuarioRepository (con findByCedula), RolRepository.
- DTOs: LoginRequest (@NotNull cedula + password), LoginResponse (token + rol), UsuarioDTO.
- JwtUtil.java — genera y valida tokens. JwtFilter.java — intercepta cada request.
- SecurityConfig.java — define endpoints públicos (/auth/login) y protegidos.
- AuthService.java — login(), logout(), getMe(). AuthController.java — 3 endpoints.
- Prueba Postman: POST /auth/login → JWT en respuesta → guardar como variable de entorno.

FASE 2 — ms-clientes

- Entidades: Cliente.java (activo BOOLEAN), TipoDocumento.java, PreferenciaCliente.java, HistorialCliente.java.
- ClienteRepository: findByCedula(), findByActivoTrue().
- ClienteService: crear(), actualizar(), buscarPorCedula(), listar(), desactivar() (soft delete).
- ClienteController: 6 endpoints con @PreAuthorize.
- Prueba: crear → buscar por cédula → verificar activo=false en soft delete vía pgAdmin.

FASE 3 — ms-unidades

- Entidades: TipoUnidad, EstadoUnidad, Unidad (@Version para optimistic locking), TipoTarifa, Tarifa.
- UnidadService: buscarDisponibles(fechaInicio, fechaFin) — query SQL con NOT IN sobre reservas activas.
- Prueba crítica: verificar que FUERA_DE_SERVICIO y MANTENIMIENTO son estados distintos.

FASE 4 — ms-reservas

- Entidades: CanalReserva, EstadoReserva, PoliticaCancelacion, Reserva (@Version), ReservaHuesped.
- ReservaService.crearReserva(): (1) verificar disponibilidad vía FeignClient, (2) crear PENDIENTE, (3) manejar OptimisticLockException → 409 Conflict.
- Prueba crítica de concurrencia: dos POST simultáneos → uno confirma, otro recibe 409.

FASE 5 — ms-checkin

- CheckinService.ejecutarCheckin() — @Transactional: reserva→EN_EJECUCION · estadía ACTIVO · unidad→OCUPADA · factura ABIERTA.
- CheckoutService.ejecutarCheckout() — @Transactional: estadía→FINALIZADO · unidad→SUCIA · factura→PAGADA · tarea limpieza PENDIENTE.
- Prueba: login → crear cliente → crear reserva → check-in → check-out → verificar todos los estados en pgAdmin.

FASE 6 — ms-housekeeping

- Entidades: EstadoLimpieza (FK), TipoLimpieza, Limpieza.
- Al recibir evento de checkout, crear tarea POST_CHECKOUT automáticamente.
- Prueba: PENDIENTE → EN_PROCESO → COMPLETADA → unidad DISPONIBLE.

FASE 7 — ms-facturacion

- Entidades: EstadoFactura, Factura (folio_tipo), FacturaDetalle (GENERATED), MetodoPago, EstadoPago, Pago.
- FacturaService: calcular total (precio_noche × noches), crear línea en factura_detalle.
- Al registrar pago completo, factura.estado → PAGADA automáticamente.

FASE 8 — Frontend React

- Axios con interceptor JWT automático hacia API Gateway en :8080.
- Zustand: token, usuario, rol del usuario autenticado.
- Vistas: Login · Lista de unidades · Crear reserva · Check-in/out · Housekeeping · Factura.
- Rutas privadas (PrivateRoute) que verifican el rol antes de renderizar.
- Prueba final: flujo completo desde el navegador sin usar Postman.

17.2 Pro — Fases P-1 a P-10

Fase	Módulo	Prioridad	Dependencias Previas
P-1	ms-facturacion upgrade (IVA, folios, Night Audit)	CRÍTICA	Beta completa
P-2	ms-pagos (pagos parciales, mixtos, reembolsos)	CRÍTICA	ms-facturacion upgrade

Fase	Módulo	Prioridad	Dependencias Previas
P-3	ms-servicios (consumos, eventos)	ALTA	Beta + ms-facturacion upgrade
P-4	ms-auditoria (auditoría completa)	ALTA	Todos los ms Beta
P-5	ms-configuracion (parámetros globales)	ALTA	Todos los ms operativos
P-6	ms-inventario (stock, alertas)	MEDIA	Beta completa
P-7	ms-mantenimiento (fallas, técnicos)	MEDIA	Beta + ms-auditoria
P-8	ms-reportes (dashboard, KPIs, exportación)	MEDIA	Todos los ms operativos
P-9	ms-dian (facturación electrónica)	MEDIA	ms-facturacion + ms-pagos
P-10	ms-ota (Booking, Expedia, Airbnb)	BAJA	Beta + ms-reportes

18. Errores Comunes, Riesgos y Mitigaciones

Error	Problema	Solución en FACILE
Reserva mal diseñada	1 habitación por reserva impide reservas grupales	reserva_huesped N:M permite múltiples huéspedes
Factura y pago mezclados	Método de pago en reserva impide pagos parciales	Relación factura 1→N pagos con tabla pago independiente
Sin estados del sistema	Habitaciones ocupadas sin reserva activa rastreable	Estados definidos como FK para reserva, unidad, estadía y factura
Sin transacciones	Fallo a mitad del check-in deja datos inconsistentes	@Transactional en operaciones críticas — rollback automático
Sin auditoría	Imposible saber quién canceló una reserva	Tabla auditoria con valor_anterior y valor_nuevo
Sin concurrencia	Dos recepcionistas reservan la misma habitación	version INT + @Version JPA (optimistic locking) → 409 Conflict
Sin contratos de API	El equipo implementa endpoints inconsistentes	Sección 10 define paths, métodos y formatos estándar
Sin Docker	Entorno no reproducible entre máquinas del equipo	docker-compose.yml levanta toda la infraestructura
estado VARCHAR	Errores de texto ('activo' vs 'ACTIVO') en estadía	Tabla estado_estadia con FK — corregido en v3.0
estado_limpieza VARCHAR	Misma inconsistencia en módulo de limpieza	Tabla estado_limpieza con FK — corregido en v4.0

18.1 Riesgos en Producción y Mitigaciones

Riesgo	Probabilidad	Mitigación
Overbooking accidental	Media	@Version JPA + verificación de disponibilidad antes de confirmar reserva
Errores en facturación	Baja	Bean Validation en DTOs + backups diarios automáticos + columna GENERATED
Pérdida de datos	Muy baja	Volúmenes Docker persistentes + backups diarios + soft delete en producción
Brecha de seguridad	Baja	Spring Security + JWT + bcrypt + permisos por rol + .env en .gitignore
No-Show sin penalización	Alta	politica_cancelacion define reglas automáticas de cobro y penalidad
Fallo de microservicio	Media	Aislamiento por contenedor — fallo de ms-reportes no afecta ms-reservas
Credenciales expuestas	Baja si se sigue el proceso	.env en .gitignore desde el primer commit. Rotación periódica de JWT_SECRET.

19. Checklist de Verificación

19.1 Verificación del Entorno — Beta Lista

Verificación	Comando	Resultado Esperado
Java instalado	java -version	openjdk 17.x.x o 21.x.x
Node.js instalado	node -v	v18.x.x o v20.x.x
Docker instalado	docker compose version	Docker Compose version v2.x
Todos los contenedores activos	docker compose ps	11+ contenedores en estado 'running'
BDs creadas	pgAdmin → cada BD	Tablas creadas correctamente en las 7 BDs
Swagger accesible	Abrir navegador	http://localhost:8081/swagger-ui.html carga la UI

19.2 Checklist de Flujo Operativo Completo

Paso	Acción	Resultado Esperado
1	POST /auth/login con cédula + password	Token JWT en la respuesta
2	POST /clientes — crear huésped	Cliente creado, activo=true en db_clientes
3	GET /unidades/disponibles con fechas	Lista de unidades disponibles
4	POST /reservas — crear reserva	Reserva en estado PENDIENTE
5	POST /checkin/{reservaId}	Estadía ACTIVO · unidad OCUPADA · factura ABIERTA
6	POST /checkout/{estadiaId}	Estadía FINALIZADO · factura PAGADA · tarea limpieza PENDIENTE
7	PATCH /limpieza/{id}/estado → COMPLETADA	Unidad → DISPONIBLE
8	GET /facturas/reserva/{id}	Factura con total correcto de noches

19.3 Pruebas de Robustez

- Concurrencia: dos POST simultáneos a la misma unidad → uno confirma (200), el otro recibe 409 Conflict.
- Soft delete: DELETE en cliente cambia activo=false, el registro existe en la BD.
- Rollback: simular fallo en ms-facturacion durante check-in → reserva vuelve a CONFIRMADA.
- Token expirado: usar JWT con fecha pasada → todos los microservicios responden 401 Unauthorized.
- Autorización por rol: recepcionista intenta acceder a endpoint de ADMIN → 403 Forbidden.

20. Glosario Técnico

Término	Definición
PMS	Property Management System. Sistema integral de gestión hotelera que centraliza reservas, check-in/out, facturación y housekeeping.
Microservicio	Servicio pequeño e independiente con su propia base de datos, desplegable de forma autónoma. Cada ms-* en FACILE es un microservicio.
API Gateway	Punto de entrada único que enruta peticiones a los microservicios correctos. En FACILE: Spring Cloud Gateway en el puerto 8080.
JWT	JSON Web Token. Estándar para transmitir identidad y permisos de forma segura entre servicios sin necesidad de consultar la BD.
BCrypt	Algoritmo de hash unidireccional para contraseñas. Incluye salt automático y es resistente a ataques de fuerza bruta.
Optimistic Locking	Mecanismo que detecta conflictos de concurrencia comparando la versión del registro. Si dos transacciones modifican el mismo registro, la segunda recibe OptimisticLockException.
Soft Delete	Marcar un registro como inactivo (activo=false) en vez de eliminarlo. Los datos se conservan para auditoría e historial.
@Transactional	Anotación de Spring que garantiza que todas las operaciones en el método se ejecutan como una unidad atómica. Si falla cualquier paso, se revierte todo.
FeignClient	Librería de Spring Cloud que permite hacer llamadas REST a otros microservicios como si fueran métodos Java locales.
Overbooking	Error donde dos reservas activas se asignan a la misma unidad para las mismas fechas. Prevenido en FACILE con optimistic locking.
CUFE	Código Único de Factura Electrónica. Identificador asignado por la DIAN al validar una factura electrónica en Colombia.
OTA	Online Travel Agency. Plataformas como Booking.com, Expedia o Airbnb que intermedian reservas hoteleras.
Night Audit	Proceso contable automático de cierre del día operativo hotelero. En FACILE implementado con @Scheduled de Spring Boot.
DDL	Data Definition Language. Sentencias SQL que definen la estructura de la base de datos (CREATE TABLE, CREATE INDEX).
JPA	Java Persistence API. Estándar Java para mapear objetos Java a tablas de base de datos relacional. Hibernate es la implementación usada.
Docker Compose	Herramienta que define y ejecuta aplicaciones multi-contenedor mediante un archivo YAML declarativo.
Bean Validation	Estándar Java (Jakarta Validation) para validar datos mediante anotaciones como @NotNull, @Email, @Size en clases Java.
Folio	Término hotelero para la factura de un huésped. Puede segmentarse en PRINCIPAL, EMPRESA y PERSONAL (como en OPERA).

21. Roles del Sistema y Permisos

Rol	Fase	Módulos Accesibles	Restricciones
ADMIN	Beta	Todos los módulos	Sin restricciones. Puede desactivar usuarios y clientes.
RECEPCION	Beta	Clientes, Unidades, Reservas, Check-in/out, Facturación básica	No puede acceder a configuración ni auditoría.
HOUSEKEEPING	Beta	Solo módulo de limpieza	Solo puede ver y actualizar tareas de limpieza.
CONTABILIDAD	Pro	Facturación completa, Pagos, Reportes financieros, Auditoría	Solo lectura en módulos no financieros.
GERENCIA	Pro	Dashboard completo, Todos los reportes, Auditoría, Configuración	Lectura total + acceso a reportes. No opera reservas.
MANTENIMIENTO	Pro	Módulo de mantenimiento, Cambio de estado de unidades	Solo gestión de fallas y mantenimiento.

Los roles se almacenan en la tabla rol y se asignan mediante la tabla N:M usuario_rol. Para agregar los roles Pro no se requiere cambios de código — solo un INSERT en la tabla rol de db_auth.